

TrainingMug AI Development Framework (ADF) v1.0

Version: 1.0

Author: TrainingMug

Purpose: Standard Software Development Framework for AI-Assisted Project Development

1. Vision

The AI Development Framework (ADF) is a standardized methodology for building production-ready software using AI coding assistants such as Freebuff, ChatGPT, Cursor, Claude Code, GitHub Copilot, or similar tools.

The framework ensures that every student develops software using professional engineering practices instead of relying on AI to generate random code.

The objective is to teach students to think like Software Architects, System Designers, and Full Stack Engineers.

2. Core Principles

Principle 1

AI is an Engineering Assistant, not a Replacement for the Engineer.

Principle 2

Design Before Development.

Principle 3

Document Before Code.

Principle 4

Build One Module at a Time.

Principle 5

Every Feature Must Be Tested.

Principle 6

Every Feature Must Be Documented.

Principle 7

Every Day Ends with a GitHub Push.

Principle 8

Every Project Must Be Deployable.

3. Standard Project Structure

```
project-name/  
  
docs/  
  
01_PROJECT_CONTEXT.md  
  
02_REQUIREMENTS.md  
  
03_ARCHITECTURE.md  
  
04_DATABASE.md  
  
05_API_CONTRACT.md  
  
06_UI_FLOW.md  
  
07_TASKS.md  
  
08_CODING_STANDARDS.md  
  
09_TESTING.md  
  
10_DEPLOYMENT.md  
  
11_PROMPTS.md
```

```
backend/  
frontend/  
docker/  
postman/  
.github/  
README.md
```

4. Development Lifecycle

Phase 1

Project Understanding

Deliverables

- Problem Statement
 - Business Objective
 - Scope
 - Assumptions
-

Phase 2

Project Context

Document

PROJECT_CONTEXT.md

Contains

- Technology Stack
- Coding Standards
- Architecture
- Folder Structure
- Branching Strategy
- Deployment Strategy

Phase 3

Requirements Engineering

Document

REQUIREMENTS.md

Includes

- Functional Requirements
- Non Functional Requirements
- User Roles
- User Stories
- Acceptance Criteria
- Out of Scope Features

Phase 4

Architecture Design

Document

ARCHITECTURE.md

Includes

- High Level Architecture
- Microservice Diagram
- Component Diagram
- Service Responsibilities
- Authentication Flow
- Deployment Diagram

Phase 5

Database Design

Document

DATABASE.md

Includes

- ER Diagram
- MySQL Schema
- Mongo Collections
- Relationships
- Constraints
- Indexes

Phase 6

API Design

Document

API_CONTRACT.md

Every API must contain

- Endpoint
- HTTP Method
- Description
- Authentication
- Request Body
- Query Parameters
- Path Variables
- Success Response
- Error Response
- Validation Rules
- Status Codes
- Sample JSON

No backend implementation should begin before API approval.

Phase 7

UI Design

Document

UI_FLOW.md

Includes

- Navigation Flow
 - Screen List
 - Dashboard Layout
 - User Journey
 - Component Hierarchy
-

Phase 8

Task Breakdown

Document

TASKS.md

Each task contains

- Task ID
 - Description
 - Priority
 - Estimated Time
 - Dependencies
 - Deliverables
 - Definition of Done
-

Phase 9

Backend Development

Develop one service at a time.

Recommended order

1. Authentication
2. User
3. Product
4. Cart
5. Wishlist
6. Order
7. Payment
8. Notification

Each service must include

- Entity
 - DTO
 - Repository
 - Service
 - Controller
 - Validation
 - Exception Handling
 - Swagger
 - Unit Tests
-

Phase 10

Frontend Development

Recommended order

- Authentication
- Dashboard
- Product Listing
- Product Details
- Wishlist
- Cart
- Checkout
- Order History
- Admin Dashboard

Every page must include

- Loading State
 - Error State
 - Empty State
 - Form Validation
 - Responsive Design
-

Phase 11

Testing

Testing Checklist

- Unit Testing
- Integration Testing

- API Testing
- UI Testing
- End-to-End Verification
- Manual Testing

Deliverables

- Test Report
 - Postman Collection
 - Bug List
-

Phase 12

Containerization

Deliverables

- Dockerfile
 - Docker Compose
 - Environment Variables
 - Production Configuration
-

Phase 13

CI/CD

Pipeline

GitHub Push

↓

Build

↓

Run Tests

↓

Build Docker Images

↓

Push Images



Deploy



Health Check

Phase 14

Deployment

Supported Platforms

Frontend

- Azure Static Web Apps
- Vercel

Backend

- Azure Container Apps
- Azure App Service

Database

- Azure MySQL
 - MongoDB Atlas
-

5. Daily Engineering Workflow

Every day follows the same routine.

1. Pull latest code.
2. Read assigned task.
3. Review project documents.
4. Create a feature branch.
5. Use AI to implement only the assigned feature.
6. Test locally.
7. Update documentation if APIs or behavior changed.
8. Commit using meaningful commit messages.

9. Push to GitHub.
 10. Create a Pull Request.
 11. Participate in code review.
 12. Merge only after approval.
-

6. Git Workflow

```
main  
  
develop  
  
feature/login  
  
feature/products  
  
feature/orders  
  
feature/payment
```

Commit Examples

```
feat: add login API  
  
feat: implement product CRUD  
  
fix: resolve JWT validation issue  
  
docs: update API contract  
  
test: add authentication unit tests  
  
refactor: optimize product service
```

7. AI Prompting Rules

Every AI session begins with:

- Read PROJECT_CONTEXT.md.
- Read REQUIREMENTS.md.
- Read API_CONTRACT.md.

- Follow CODING_STANDARDS.md.
 - Generate only the requested module.
 - Never modify unrelated modules.
 - Explain assumptions before coding.
 - Stop after completing the requested task.
-

8. Coding Standards

Mandatory

- SOLID Principles
- Layered Architecture
- DTO Pattern
- Repository Pattern
- Constructor Injection
- Validation
- Global Exception Handler
- Swagger Documentation
- REST Naming Standards
- JUnit Testing
- Meaningful Logging

Avoid

- Business logic in Controllers
 - Exposing Entities
 - Hardcoded Values
 - Duplicate Code
 - Long Methods
 - Magic Numbers
-

9. Definition of Done

A feature is complete only when:

- Business requirement implemented.
- UI completed.
- Backend completed.
- API documented.
- Validation added.
- Exception handling implemented.
- Unit tests passed.
- Manual testing completed.

- Code pushed to GitHub.
 - Pull Request approved.
 - Documentation updated.
-

10. Deliverables Checklist

Every project submission must include:

- Source Code
 - GitHub Repository
 - README
 - Project Context
 - Requirements Document
 - Architecture Document
 - Database Design
 - API Contract
 - Swagger Documentation
 - Postman Collection
 - Docker Configuration
 - CI/CD Pipeline
 - Deployment URL
 - Screenshots
 - Demo Video
 - Presentation
-

11. Evaluation Rubric (100 Marks)

Category	Marks
Requirements & Analysis	10
Architecture & Design	10
Database Design	10
Backend Development	15
Frontend Development	15
Security	10
API Documentation	5
Testing	10
Git & Collaboration	5

Category	Marks
Docker & CI/CD	5
Cloud Deployment	5

12. TrainingMug AI Development Pledge

Every student agrees to:

- Understand the problem before writing code.
- Use AI responsibly.
- Verify all AI-generated code.
- Maintain documentation throughout the project.
- Write clean, secure, and maintainable software.
- Commit code daily.
- Review teammates' code respectfully.
- Deliver software that is production-ready.

"Design First. Build Smart. Verify Always. Deploy with Confidence."